

PROGRAMMIERUNG VON WEB-ANWENDUNGEN:

FRONTEND-BACKEND-INTEGRATION (REST)

WAS IST OPENAPI UND WARUM NUTZEN WIR ES?

- OpenAPI ist ein Standard zur Beschreibung von REST-APIs in maschinenlesbarer Form
- Die Spezifikation wird meist in YAML geschrieben
- Mit OpenAPI lassen sich automatisch Client- und Servercode generieren
- Ziel: weniger manueller Aufwand und klar definierte Schnittstellen
- Wir verwenden es, um React und Spring Boot sauber zu koppeln

OPENAPI-DOKUMENT ERSTELLEN (SCHRITT 1)

- Die API-Beschreibung erfolgt in einer YAML-Datei namens openapi.yaml
- Man kann sie manuell schreiben oder über Tools erzeugen
- Im Beispiel wird eine einfache GET-Anfrage zur Abfrage einer Nachricht beschrieben
- Es werden Pfad, HTTP-Methode, Rückgabetyt und Beschreibung angegeben
- Die Datei liegt typischerweise im Root-Verzeichnis des Projekts

yaml

```
openapi: 3.0.0
info:
  title: Hello API
  version: 1.0.0
paths:
  /hello:
    get:
      summary: Gibt eine Begrüßung zurück
      responses:
        '200':
          description: Erfolgreiche Antwort
          content:
            text/plain:
              schema:
                type: string
```

CODE MIT OPENAPI GENERATOR ERZEUGEN (SCHRITT 2)

- OpenAPI Generator erzeugt automatisch TypeScript- oder Java-Code aus der API-Beschreibung
- Für React wählen wir den Client-Typ typescript-fetch
- Der Generator erstellt Methoden, Schnittstellen und API-Aufrufe passend zur Spezifikation
- Voraussetzung ist eine korrekt geschriebene openapi.yaml
- Der Befehl kann lokal oder über Tools wie SwaggerHub ausgeführt werden

bash

```
openapi-generator-cli generate \  
  -i openapi.yaml \  
  -g typescript-fetch \  
  -o ./generated-api
```

INTEGRATION DES GENERIERTEN CLIENTS IN REACT (SCHRITT 3)

- Der generierte Code wird in das React-Projekt kopiert oder eingebunden
- Die API-Funktionen können direkt in React-Komponenten verwendet werden
- Importierte Funktionen wie getHello nutzen automatisch die korrekte URL und Struktur
- Es müssen keine manuellen fetch-Aufrufe geschrieben werden
- Die Nutzung ist typsicher und erleichtert Fehlersuche und Wartung

tsx

```
import { getHello } from './generated-api';

useEffect(() => {
  getHello()
    .then((response) => {
      console.log(response); // "Hallo aus dem Backend!"
    })
    .catch((error) => {
      console.error("Fehler beim API-Aufruf:", error);
    });
}, []);
```

BEISPIELHAFTER EINSATZ IN EINER REACT-KOMPONENTE

- Die generierte Methode wird z.B. in useEffect eingebunden, um Daten zu laden
- Die Antwort kann direkt im State gespeichert und angezeigt werden
- Typdefinitionen aus dem Generator helfen bei der Verwendung im Code
- Fehlerbehandlung und Ladeanzeige können wie gewohnt eingebaut werden
- Das API-Verhalten bleibt konsistent mit dem Backend ohne doppelten Code

tsx

```
import React, { useEffect, useState } from 'react';
import { getHello } from './generated-api';

const HelloComponent = () => {
  const [message, setMessage] = useState("");

  useEffect(() => {
    getHello()
      .then((data) => setMessage(data))
      .catch((err) => console.error(err));
  }, []);

  return <h1>{message}</h1>;
};

export default HelloComponent;
```

VORTEILE DER OPENAPI-INTEGRATION

- Vermeidung von doppeltem Code im Frontend und Backend
- Automatisch generierter Code ist konsistent und reduziert Fehler
- Schnellere Entwicklung durch weniger manuelle Schritte
- Typensicherheit und bessere Wartbarkeit in React
- Klare Trennung und Dokumentation der Schnittstellen

ZUSAMMENFASSUNG UND AUSBLICK

- OpenAPI ermöglicht eine klare und zentrale Definition der REST-Schnittstellen
- Durch Code-Generierung sparen wir Zeit und vermeiden Fehler
- Frontend und Backend bleiben automatisch synchron
- Die Integration in React ist einfach und typsicher
- Im nächsten Schritt könnt ihr eigene APIs auf diese Weise einbinden

[Demo / Umsetzung](#)